

# Estudio de Técnicas de Aprendizaje Automático usando Sobremuestreo de Datos

Autor: Aarón Zeballo  
Tutora: Dra. Andrea Rey

Universidad Nacional de Hurlingham (UNAHUR)



# Índice general

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>Índice de Figuras</b>                                           | <b>2</b>  |
| <b>Índice de Tablas</b>                                            | <b>3</b>  |
| <b>1. Introducción</b>                                             | <b>6</b>  |
| 1.1. Introducción . . . . .                                        | 6         |
| 1.1.1. Orígenes y Evolución del Correo No Deseado . . . . .        | 6         |
| 1.1.2. El Desafío del Desbalanceo de Clases . . . . .              | 8         |
| 1.1.3. Motivación y Objetivo General . . . . .                     | 8         |
| 1.2. Estado del Arte . . . . .                                     | 9         |
| 1.2.1. Tratamiento de datos desbalanceados . . . . .               | 9         |
| 1.3. Conjunto de Datos . . . . .                                   | 10        |
| 1.3.1. Estructura de las Variables Predictoras . . . . .           | 11        |
| 1.3.2. El Problema de las Escalas . . . . .                        | 12        |
| <b>2. Metodología</b>                                              | <b>13</b> |
| 2.1. Técnicas de sobremuestreo . . . . .                           | 14        |
| 2.2. Modelos de Clasificación . . . . .                            | 15        |
| <b>3. Resultados</b>                                               | <b>19</b> |
| 3.1. Resultados de las técnicas de sobremuestreo . . . . .         | 20        |
| 3.2. Comparación de datos sintéticos frente a originales . . . . . | 22        |
| <b>4. Conclusiones</b>                                             | <b>24</b> |
| <b>Bibliografía</b>                                                | <b>26</b> |

# Índice de Figuras

|                                                                                                                            |    |
|----------------------------------------------------------------------------------------------------------------------------|----|
| 2.1. Distribución de clases: Conjunto Original, Desbalanceado (95/5) y Sobremuestreado (70/30). . . . .                    | 13 |
| 3.1. Resultados del sobremuestreo al 50/50: <i>Accuracy</i> , <i>Precision</i> , <i>F1-Score</i> y <i>Recall</i> . . . . . | 20 |
| 3.2. Comparación final de métricas utilizando la proporción óptima (70/30). . . . .                                        | 21 |
| 3.3. Comparación de métricas: Conjunto Original vs. Sobremuestreo ADASYN (70/30). . . . .                                  | 23 |

# Índice de Tablas

|                                                                                                             |    |
|-------------------------------------------------------------------------------------------------------------|----|
| 1.1. Descripción de las variables del conjunto de datos Spam Base . . . . .                                 | 10 |
| 2.1. Parámetros predeterminados utilizados en los clasificadores basados en árboles (scikit-learn). . . . . | 16 |
| 2.2. Parámetros predeterminados utilizados en la Máquina de Soporte Vectorial (scikit-learn). . . . .       | 16 |
| 2.3. Grilla de hiperparámetros evaluados y ajustados para el modelo SVM. . . . .                            | 18 |
| 3.1. Métricas obtenidas mediante el clasificador SVM (ADASYN) justificando la proporción 70/30. . . . .     | 21 |

# Resumen

Los modelos de aprendizaje automático se enfrentan al desequilibrio de clases; este desequilibrio perjudica el aprendizaje de los algoritmos, los cuales tienden a ignorar la clase minoritaria y pierden capacidad predictiva. Para tratar de solucionar esta problemática, el presente trabajo se centró en el estudio de distintos algoritmos de sobremuestreo de datos como solución para equilibrar las clases. Se evaluaron técnicas clásicas como SMOTE y ADASYN, así como sus evoluciones modernas, poniéndolas a prueba con una problemática importante en la ciberseguridad: la detección de correos no deseados (*spam*). Para medir su efectividad se utilizaron tres clasificadores: Árbol de Decisión, *Random Forest* y Máquinas de Soporte Vectorial (SVM).

Los resultados mostraron un buen desempeño por parte de las técnicas de sobremuestreo para contrarestar el desbalanceo en las clases. Específicamente, el método ADASYN combinado con SVM tuvo el mejor rendimiento a la hora de detectar *spam*, logrando alcanzar un *Recall* de 0.86 y un *F1-Score* de 0.75 sobre la clase minoritaria. Esto demostró que, a partir de un conjunto de datos fuertemente desbalanceado y posteriormente sobremuestreado, es posible obtener resultados predictivos muy cercanos a los que se pueden obtener a partir de un conjunto de datos con mayor equilibrio entre clases y ampliamente superior a los de un conjunto de datos desbalanceado.

# Capítulo 1

## Introducción

### 1.1 Introducción

La comunicación a través del correo electrónico se ha consolidado como una herramienta indispensable en el ámbito personal y corporativo. Sin embargo, su masificación trajo consigo una amenaza recurrente y con alto impacto económico de la era digital: el correo no deseado o *spam*.

Este capítulo expone el recorrido histórico sobre el origen y la evolución de esta problemática, detallando cómo pasó de ser una simple molestia publicitaria a convertirse en uno de los principales causantes de ataques cibernéticos. Posteriormente, se analizará el desafío fundamental que enfrentan los sistemas de defensa modernos al intentar clasificar este tráfico utilizando Aprendizaje Automático debido al desbalanceo inherente de los datos, para finalmente establecer los objetivos generales de este trabajo de investigación.

#### 1.1.1 Orígenes y Evolución del Correo No Deseado

Históricamente, el primer incidente documentado de correo electrónico no deseado, *spam*, ocurrió el 3 de mayo de 1978. Gary Thuerk, un gerente de marketing de la empresa Digital Equipment Corporation (DEC), envió un mensaje promocionando un nuevo modelo de computadora a casi 400 usuarios de ARPANET (la red precursora de internet, en ese entonces de uso restringido) [13]. Esta acción, que consistió en escribir manualmente cientos de direcciones, generó un fuerte rechazo entre la comunidad científica de la época, pero sentó el precedente de lo que más tarde se convertiría en una industria masiva.

Un hito tecnológico relevante ocurrió en abril de 1994, cuando un par de abogados estadounidenses utilizaron un programa para enviar publicidad masiva sobre la lotería de visas de Estados Unidos a miles de grupos de noticias en la red *Usenet*. Este evento, conocido históricamente como el “Green Card Spam” [14], demostró al mundo que, gracias a la automatización, un individuo podía contactar a millones de personas a un costo casi nulo, marcando el nacimiento del *spam* comercial moderno.

Con la popularización de internet a fines de la década de los 90 y principios de los 2000, el ecosistema del correo basura evolucionó hacia la ciberdelincuencia. Los atacantes dejaron de usar sus propios servidores para evitar ser rastreados, y comenzaron a utilizar *botnets*. Las *botnets*

son redes globales compuestas por miles o millones de computadoras personales de usuarios comunes que han sido previamente infectadas en secreto por un virus. Los atacantes controlan estas computadoras a distancia para que envíen millones de mensajes por minuto, volviendo la amenaza indetectable y masiva.

Actualmente, el correo no deseado es un problema de seguridad informática crítico. La amenaza ha madurado considerablemente: ya no se trata solamente de publicidad molesta o correos irrelevantes, sino que es la herramienta principal del cibercrimen organizado. A través de estos correos se distribuyen constantemente ataques de secuestro de datos (*ransomware*), intentos de robo de contraseñas (*phishing*) y diversos programas maliciosos (*malware*).

El impacto de este problema afecta gravemente en dos niveles principales:

A nivel corporativo y a gran escala, el *spam* inunda las redes empresariales. Esto no solo satura el ancho de banda y el almacenamiento de los servidores de las compañías, sino que genera una pérdida significativa de tiempo operativo al personal de soporte técnico que debe lidiar con la basura digital. Adicionalmente, existe el riesgo de que un empleado abra un archivo adjunto infectado en un correo disfrazado, permitiendo que los ciberdelincuentes vulneren las defensas y accedan a datos privados de la empresa, desencadenando pérdidas millonarias o extorsiones.

En el ámbito personal, los usuarios comunes son los más vulnerables. Diariamente se enfrentan a correos engañosos diseñados con ingeniería social para parecer alertas de sus bancos o mensajes de la administración pública. Caer en estos engaños puede resultar en la sustracción no autorizada de fondos, fraudes con tarjetas de crédito, robo de identidad o la inutilización total de sus dispositivos personales a causa de virus destructivos.

Para ejemplificar la problemática, se considera oportuno mostrar ejemplos recientes que demuestran el peligro al que se exponen las empresas y usuarios si un correo malicioso pasa los filtros de seguridad.

Un caso emblemático es el de la red Emotet, considerada por Europol como el *malware* más peligroso del mundo hasta su desmantelamiento en 2021 [9]. Emotet operaba principalmente a través de campañas automatizadas de *spam*. Los atacantes enviaban correos electrónicos que simulaban ser facturas legítimas, notificaciones de envíos o alertas de bancos conocidos, que contenían documentos infectados. Una vez que el usuario abría el archivo, el virus se instalaba en silencio, permitiendo a los delincuentes robar credenciales bancarias o instalar *ransomware* en toda la red de la empresa. Este método de propagación por correo causó daños globales estimados en cientos de millones de dólares.

Por otro lado, durante los últimos años, Argentina ha sufrido oleadas masivas de campañas de *phishing* mediante correos electrónicos diseñados para robar contraseñas de plataformas de uso cotidiano, siendo el caso de Mercado Pago uno de los más notorios y destructivos [8].

En esta modalidad, los ciberdelincuentes envían correos falsos avisando a los usuarios que su cuenta ha sido bloqueada por actividad sospechosa. Debido al miedo a perder su dinero, la víctima hace clic en el enlace proporcionado, el cual la dirige a una página web visualmente idéntica a la original. Al ingresar sus datos, los atacantes toman el control total de la billetera virtual. Esto resulta en innumerables casos de vaciado de cuentas e incluso en la toma de préstamos a nombre de las víctimas. Este tipo de fraude genera daños financieros en familias, las cuales sufren la pérdida de sus ahorros o quedan endeudadas por un correo electrónico que no fue filtrado correctamente.

### 1.1.2 El Desafío del Desbalanceo de Clases

Los métodos de filtrado tradicionales, basados en reglas fijas como bloquear correos que contengan la palabra “gratis” o rechazar conexiones de ciertas direcciones IP, quedaron rápidamente obsoletos. Los atacantes encontraron formas sencillas de esquivar estos bloqueos alterando la escritura de las palabras. Por este motivo, la industria de la ciberseguridad se volcó hacia el Aprendizaje Automático (*Machine Learning*), utilizando algoritmos dinámicos capaces de identificar patrones tanto en el texto como en la estructura de los correos. [6]

Pero al intentar entrenar estos algoritmos, surge un gran problema: el desbalanceo natural de los datos. Ya sea en una cuenta empresarial o en una cuenta de correo electrónico personal, la mayoría de los correos que se reciben diariamente son legítimos. Los correos maliciosos representan solo una minoría.

Cuando le entregamos esta información desbalanceada a un algoritmo para que aprenda, el modelo, al tratar de tener el mayor porcentaje de aciertos suele ignorar la clase minoritaria, terminando con valores altos en aciertos totales pero mostrando un rendimiento deficiente en la detección y separación de los ejemplos de la clase minoritaria, dejando pasar correos maliciosos y potencialmente peligrosos para los usuarios.

### 1.1.3 Motivación y Objetivo General

Identificar y filtrar los correos maliciosos se debe hacer de forma automática y con un margen mínimo, ya que los dos tipos de errores posibles tienen consecuencias graves. Si el filtro es demasiado estricto y marca un correo normal como *spam* (Falso Positivo), un usuario podría perder un correo de trabajo urgente, una oferta laboral o el aviso de vencimiento de un servicio. Por el contrario, si el filtro es demasiado laxo y deja pasar un correo peligroso tratándolo como legítimo (Falso Negativo), las consecuencias van desde el vaciamiento de una cuenta bancaria personal hasta la infección de toda una red empresarial por un virus.

El problema es que existe una balanza entre estos dos errores: cuando buscamos que el filtro atrape absolutamente todas las amenazas, el sistema empieza a tratar correos electrónicos legítimos como si fueran correos no deseados. Encontrar el punto de equilibrio exacto entre detectar *spam* y que los usuarios no pierdan correos importantes resulta un factor determinante.

El trabajo se propone comparar cómo funcionan diferentes modelos de Aprendizaje Automático frente a un conjunto de datos que ha sido equilibrado mediante la creación de ejemplos sintéticos artificiales.

Para alcanzar esta meta, se establecerán los siguientes objetivos:

- Implementar y comparar el rendimiento de tres clasificadores: Árbol de Decisión, Random Forest y Máquinas de Soporte Vectorial (SVM).
- Evaluar distintas técnicas de balanceo de datos, partiendo desde el clásico SMOTE, pasando por ADASYN enfocado en las fronteras de decisión y métodos más modernos que buscan ser la evolución de los anteriores, tales como G-SMOTE y ADASYN + ENN.
- Comparar y mostrar resultados a través de métricas robustas frente al desbalanceo tales como *recall* y *F1-Score*.

## 1.2 Estado del Arte

### 1.2.1 Tratamiento de datos desbalanceados

El problema del desequilibrio de clases no es nuevo. A principios de la década de los 2000, las primeras estrategias para intentar resolver este sesgo eran bastante básicas y se realizaban directamente sobre los datos crudos. Las dos técnicas principales consistían en borrar al azar ejemplos de la clase mayoritaria (conocido como *Random Undersampling*) o simplemente duplicar los correos de la clase minoritaria copiándolos exactamente iguales varias veces (*Random Oversampling*). Tal como recoge Chawla en sus investigaciones [4], estas técnicas presentaban limitaciones técnicas considerables. El submuestreo provocaba una pérdida masiva de información útil, ya que se borraban correos legítimos que el modelo necesitaba para aprender. Por su parte, la duplicación exacta de datos causaba un fenómeno de sobreajuste (*overfitting*); el modelo no aprendía a generalizar los patrones de un ataque, sino que se limitaba a memorizar de forma exacta los pocos correos maliciosos repetidos.

El verdadero cambio de paradigma en el manejo de datos desbalanceados ocurrió en 2002 con la publicación de *SMOTE: Synthetic Minority Over-sampling Technique* por Nitesh V. Chawla y su equipo [4]. En lugar de copiar ejemplos de la clase minoritaria, SMOTE introdujo la idea de “crear” nuevos ejemplos que tuvieran características extraídas desde los ejemplos originales. Para lograrlo, el algoritmo toma un ejemplo de la clase minoritaria, busca a sus vecinos más cercanos y traza líneas rectas entre ellos para generar nuevos puntos artificiales a lo largo de esa trayectoria. Esta técnica logró expandir el territorio de la clase minoritaria de forma segura, evitando que el modelo memorizara copias exactas y convirtiéndose rápidamente en el estándar de la industria. [10]

A pesar de su éxito, SMOTE tiene una debilidad: genera datos sintéticos de manera uniforme para todos los ejemplos de la clase minoritaria. Esto significa que crea datos tanto en zonas seguras como en zonas fronterizas (donde los ejemplos de la clase minoritaria se mezclaban con los datos de la clase mayoritaria), lo que termina generando ruido. Para solucionar este defecto, Haibo He, Yang Bai, Shutao Li y Eduardo García. [11] desarrollaron en 2008 el algoritmo ADASYN (*Adaptive Synthetic*). ADASYN mejoró la propuesta de SMOTE agregando una capa de cálculo extra. El algoritmo evalúa primero qué tan difícil es de clasificar cada caso de la clase minoritaria. Luego, procede a la creación de datos sintéticos únicamente alrededor de aquellos casos que están en zonas donde hay tanto ejemplos de la clase mayoritaria como de la minoritaria. De esta forma, el algoritmo le presta mayor atención a los casos engañosos y no en las regiones que el modelo ya domina y no son conflictivas.

La búsqueda por lograr la mejora en la creación de datos sintéticos siguió evolucionando. En años recientes, investigaciones como las de Douzas y Bacao propusieron *Geometric SMOTE* (G-SMOTE) [7]. En lugar de trazar líneas rectas que pueden invadir el territorio de la clase mayoritaria, G-SMOTE restringe la generación de datos sintéticos al interior de esferas alrededor de las observaciones de la clase minoritaria, reduciendo la creación de ruido. Paralelamente, investigaciones clásicas como las de Batista [1] demostraron que, en conjuntos de datos muy complejos, crear datos no siempre es suficiente. Esto dio origen a los métodos híbridos, que combinan técnicas generativas (como SMOTE) con algoritmos de limpieza (como *Edited Nearest*

*Neighbours*, ENN). En este enfoque, primero se crean datos sintéticos de la clase minoritaria y luego se utilizan algoritmos de limpieza para borrar todos los ejemplos que hayan quedado mal posicionados. Este doble esfuerzo produce zonas de decisión mejores.

### 1.3 Conjunto de Datos

Para el desarrollo de este trabajo se utilizó el conjunto de datos público *Spam Email Classification* <https://www.kaggle.com/datasets/somesh24/spambase>. La clase positiva (*spam*) en este conjunto de datos incluye anuncios de productos no solicitados, correos con potencial de estafa y contenido inapropiado. Por otro lado, la clase negativa (correos legítimos) proviene directamente del ámbito laboral y personal de los creadores originales del conjunto de datos. El conjunto de datos original está estructurado en 4601 registros (correos electrónicos) y 58 variables (detalladas en la Tabla 1.1). De estas, 57 actúan como variables predictoras continuas (características extraídas mediante procesamiento de texto) y la última corresponde a la variable objetivo (*target*) indicando si es un correo legítimo o de **spam**.

Tabla 1.1: Descripción de las variables del conjunto de datos Spam Base

| Índice                             | Nombre de la Variable | Descripción                              |
|------------------------------------|-----------------------|------------------------------------------|
| 1                                  | word_freq_make        | Frecuencia de la palabra “make” (%)      |
| 2                                  | word_freq_address     | Frecuencia de la palabra “address” (%)   |
| 3                                  | word_freq_all         | Frecuencia de la palabra “all” (%)       |
| 4                                  | word_freq_3d          | Frecuencia de la palabra “3d” (%)        |
| 5                                  | word_freq_our         | Frecuencia de la palabra “our” (%)       |
| 6                                  | word_freq_over        | Frecuencia de la palabra “over” (%)      |
| 7                                  | word_freq_remove      | Frecuencia de la palabra “remove” (%)    |
| 8                                  | word_freq_internet    | Frecuencia de la palabra “internet” (%)  |
| 9                                  | word_freq_order       | Frecuencia de la palabra “order” (%)     |
| 10                                 | word_freq_mail        | Frecuencia de la palabra “mail” (%)      |
| 11                                 | word_freq_receive     | Frecuencia de la palabra “receive” (%)   |
| 12                                 | word_freq_will        | Frecuencia de la palabra “will” (%)      |
| 13                                 | word_freq_people      | Frecuencia de la palabra “people” (%)    |
| 14                                 | word_freq_report      | Frecuencia de la palabra “report” (%)    |
| 15                                 | word_freq_addresses   | Frecuencia de la palabra “addresses” (%) |
| 16                                 | word_freq_free        | Frecuencia de la palabra “free” (%)      |
| 17                                 | word_freq_business    | Frecuencia de la palabra “business” (%)  |
| 18                                 | word_freq_email       | Frecuencia de la palabra “email” (%)     |
| 19                                 | word_freq_you         | Frecuencia de la palabra “you” (%)       |
| 20                                 | word_freq_credit      | Frecuencia de la palabra “credit” (%)    |
| 21                                 | word_freq_your        | Frecuencia de la palabra “your” (%)      |
| 22                                 | word_freq_font        | Frecuencia de la palabra “font” (%)      |
| 23                                 | word_freq_000         | Frecuencia de la palabra “000” (%)       |
| Continúa en la siguiente página... |                       |                                          |

Tabla 1.1 – Continuación de la página anterior

| Índice | Nombre de la Variable      | Descripción                                   |
|--------|----------------------------|-----------------------------------------------|
| 24     | word_freq_money            | Frecuencia de la palabra “money” (%)          |
| 25     | word_freq_hp               | Frecuencia de la palabra “hp” (%)             |
| 26     | word_freq_hpl              | Frecuencia de la palabra “hpl” (%)            |
| 27     | word_freq_george           | Frecuencia de la palabra “george” (%)         |
| 28     | word_freq_650              | Frecuencia de la palabra “650” (%)            |
| 29     | word_freq_lab              | Frecuencia de la palabra “lab” (%)            |
| 30     | word_freq_labs             | Frecuencia de la palabra “labs” (%)           |
| 31     | word_freq_telnet           | Frecuencia de la palabra “telnet” (%)         |
| 32     | word_freq_857              | Frecuencia de la palabra “857” (%)            |
| 33     | word_freq_data             | Frecuencia de la palabra “data” (%)           |
| 34     | word_freq_415              | Frecuencia de la palabra “415” (%)            |
| 35     | word_freq_85               | Frecuencia de la palabra “85” (%)             |
| 36     | word_freq_technology       | Frecuencia de la palabra “technology” (%)     |
| 37     | word_freq_1999             | Frecuencia de la palabra “1999” (%)           |
| 38     | word_freq_parts            | Frecuencia de la palabra “parts” (%)          |
| 39     | word_freq_pm               | Frecuencia de la palabra “pm” (%)             |
| 40     | word_freq_direct           | Frecuencia de la palabra “direct” (%)         |
| 41     | word_freq_cs               | Frecuencia de la palabra “cs” (%)             |
| 42     | word_freq_meeting          | Frecuencia de la palabra “meeting” (%)        |
| 43     | word_freq_original         | Frecuencia de la palabra “original” (%)       |
| 44     | word_freq_project          | Frecuencia de la palabra “project” (%)        |
| 45     | word_freq_re               | Frecuencia de la palabra “re” (%)             |
| 46     | word_freq_edu              | Frecuencia de la palabra “edu” (%)            |
| 47     | word_freq_table            | Frecuencia de la palabra “table” (%)          |
| 48     | word_freq_conference       | Frecuencia de la palabra “conference” (%)     |
| 49     | char_freq_;                | Frecuencia del carácter “;” (%)               |
| 50     | char_freq_(                | Frecuencia del carácter “(” (%)               |
| 51     | char_freq_[                | Frecuencia del carácter “[” (%)               |
| 52     | char_freq_!                | Frecuencia del carácter “!” (%)               |
| 53     | char_freq_\$               | Frecuencia del carácter “\$” (%)              |
| 54     | char_freq_#                | Frecuencia del carácter “#” (%)               |
| 55     | capital_run_length_average | Longitud promedio de secuencias en mayúsculas |
| 56     | capital_run_length_longest | Longitud máxima de secuencia en mayúsculas    |
| 57     | capital_run_length_total   | Suma total de letras mayúsculas en el correo  |
| 58     | class                      | Variable objetivo (1 = Spam, 0 = Legítimo)    |

### 1.3.1 Estructura de las Variables Predictoras

Las características extraídas de cada correo electrónico no contienen el texto original, sino que representan métricas numéricas. Estas variables se dividen en tres grandes familias:

1. **Frecuencia de palabras (48 atributos):** `word_freq_`, estas variables miden el porcentaje de aparición de la palabra en el correo (por ejemplo: *free*, *money*, *business*). Matemáticamente, se calculan contando el número de veces que aparece la palabra, dividiéndolo por el número total de palabras en el correo, y multiplicando el resultado por 100.
2. **Frecuencia de caracteres especiales (6 atributos):** Bajo el prefijo `char_freq_`, miden el porcentaje de aparición de caracteres no alfanuméricos clave, los cuales representan predictores extremadamente fuertes de correos maliciosos.
  - `char_freq_;` → Frecuencia del punto y coma (;).
  - `char_freq_(` → Frecuencia del paréntesis de apertura (()).
  - `char_freq_[` → Frecuencia del corchete de apertura ([).
  - `char_freq_!` → Frecuencia del signo de exclamación (!).
  - `char_freq_$` → Frecuencia del signo de dólar (\$).
  - `char_freq_#` → Frecuencia del símbolo numeral o hashtag (#).
3. **Secuencias de letras mayúsculas (3 atributos):** El uso de mayúsculas (ej. “COMPRA AHORA”, “ÚLTIMA OPORTUNIDAD”) es un patrón clásico en la *spam*. Estas variables, denominadas `capital_run_length_`, se verán reflejadas a través de tres métricas:
  - *Average*: La longitud promedio de las secuencias de letras mayúsculas ininterrumpidas.
  - *Longest*: La secuencia de letras mayúsculas más larga encontrada en todo el correo.
  - *Total*: La suma total de todas las letras mayúsculas presentes en el mensaje.

### 1.3.2 El Problema de las Escalas

Al analizar los valores del conjunto de datos, surge un problema importante relacionado con las escalas numéricas. Por un lado, las variables que miden la frecuencia de palabras están expresadas en porcentajes, por lo que la mayoría de sus valores se encuentran entre 0 y 5. Por otro lado, la variable `capital_run_length_total` representa un conteo absoluto, con valores que pueden llegar a miles en un solo correo.

Esta diferencia de escalas es un problema para las Máquinas de Soporte Vectorial (SVM), dado que el SVM clasifica los datos calculando distancias geométricas (como la distancia euclidiana) entre los puntos, los números más grandes terminan dominando el cálculo. Si el modelo es entrenado con los datos en su formato original, el algoritmo le daría casi toda la importancia a la cantidad de mayúsculas solo porque sus números varían en miles.

Para solucionar esto, será necesaria una estandarización previa al entrenamiento de los modelos. Mediante el uso de herramientas como `StandardScaler`, todas las variables numéricas se ajustan matemáticamente para tener una media de cero y una varianza de uno. Al igualar las escalas, nos aseguramos de que el algoritmo evalúe de manera justa todas las variables y le dé a cada característica el peso que le corresponde.

## Capítulo 2

# Metodología

Dado que el objetivo principal de este trabajo es estudiar el comportamiento de los distintos algoritmos de aprendizaje automático frente a clases fuertemente desbalanceadas y los métodos de sobremuestreo, se procedió a eliminar aleatoriamente instancias de la clase positiva (correos maliciosos) hasta que estos representaran únicamente el 5 % del total de los datos (tal como se ilustra en la Figura 2.1) para reflejar la distribución habitual de los correos que le llegan a una persona, donde las amenazas son una minoría frente a la cantidad de correo legítimo [6].

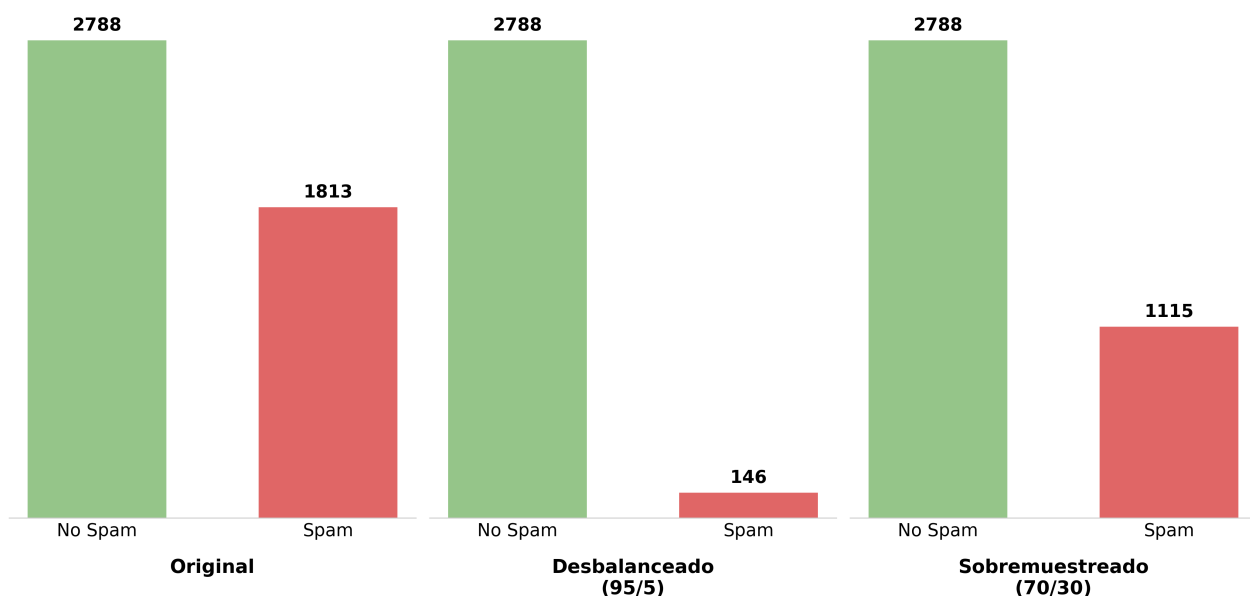


Figura 2.1: Distribución de clases: Conjunto Original, Desbalanceado (95/5) y Sobremuestreado (70/30).

A continuación, el conjunto de datos resultante se dividió en dos subconjuntos: un 80 % de las instancias se destinó al entrenamiento de los modelos, mientras que el 20 % restante se reservó exclusivamente para las pruebas de evaluación final.

Para evitar cualquier posible filtración de información (*data leakage*) desde el conjunto de

prueba hacia la fase de aprendizaje, la estandarización de las escalas numéricas se realizó de forma estricta después de la división. Se utilizó la herramienta **StandardScaler** (provista por la librería **scikit-learn** [12]) para ajustar los valores numéricos de las variables predictoras a una media de cero y varianza de uno. Este escalador se ajustó utilizando únicamente la información de los datos de entrenamiento. Posteriormente, esas mismas reglas calculadas se utilizaron tanto para el conjunto de entrenamiento como el de prueba. Este paso de estandarización es un requisito para garantizar el correcto funcionamiento de los clasificadores basados en el cálculo de distancias geométricas.

## 2.1 Técnicas de sobremuestreo

Para contrarrestar el fuerte desequilibrio de clases descrito en la sección anterior, se implementaron cuatro enfoques distintos de sobremuestreo. Estas técnicas emplean algoritmos para generar nuevas instancias sintéticas basándose en las relaciones matemáticas y geométricas de los ejemplos ya presentes en el espacio de datos. El análisis se estructuró comenzando por algoritmos generativos clásicos, para luego avanzar hacia sus respectivas evoluciones modernas.

**SMOTE (Synthetic Minority Over-sampling Technique) [4].** Es el algoritmo estándar a la hora de crear datos sintéticos para balancear un conjunto de datos. Funciona analizando los ejemplos de la clase minoritaria (los correos *spam*) uno por uno. Para cada correo, el algoritmo busca a sus  $k$  vecinos más cercanos (en el presente trabajo, serían  $k = 5$ ).

Una vez elegido un vecino al azar, la generación del nuevo dato sintético ( $\mathbf{x}_{\text{nuevo}}$ ) se realiza mediante una interpolación matemática lineal entre la instancia original ( $\mathbf{x}_i$ , donde  $i = 1, \dots, N_{\text{min}}$  recorre todas las observaciones de la clase minoritaria) y su vecino ( $\mathbf{x}_{\text{vecino}}$ ):

$$\mathbf{x}_{\text{nuevo}} = \mathbf{x}_i + \lambda \times (\mathbf{x}_{\text{vecino}} - \mathbf{x}_i) \quad (2.1)$$

donde  $\lambda$  es un número aleatorio entre 0 y 1 que ubica al nuevo correo en un punto intermedio de la línea que los une. La cantidad de vecinos que el algoritmo conecta depende de la brecha de datos que se deba llenar. Para este estudio, donde se partió de un número extremadamente bajo de clase minoritaria (apenas el 5 %), el algoritmo generó aproximadamente 7 nuevas instancias sintéticas a partir de cada correo original para alcanzar el balance deseado/[4].

**ADASYN (Adaptive Synthetic) [11].** La principal diferencia y ventaja de ADASYN frente a SMOTE es que presta mucha más atención a la frontera de decisión, es decir, a la zona de solapamiento donde los correos *spam* y los legítimos se mezclan y son difíciles de separar.

Para lograr esto, el algoritmo primero calcula la función de densidad de probabilidad de la clase mayoritaria alrededor de cada ejemplo *spam*. Con esa información, calcula un ratio de "dificultad" de aprendizaje para cada instancia. Luego, procede a la creación de datos artificiales únicamente alrededor de los correos que resultaron ser más conflictivos de clasificar. De esta manera, el modelo concentra su esfuerzo en la frontera límite, evitando generar información innecesaria en zonas seguras que los modelos ya pueden separar fácilmente.

**G-SMOTE (Geometric SMOTE) [7].** Es una evolución geométrica directa del algoritmo SMOTE original. El problema del enfoque clásico de SMOTE es que crea los datos nuevos estrictamente sobre líneas rectas, lo que puede generar ruido si esas líneas cruzan cerca de la clase contraria.

G-SMOTE soluciona esto definiendo una zona esférica alrededor de las instancias originales de la clase minoritaria, generando las nuevas muestras en cualquier punto dentro de esa región esférica. La clave que hace a este algoritmo tan seguro y preciso es cómo define el tamaño de su esfera: antes de crearla, calcula la distancia euclidiana exacta hacia el ejemplo de la clase mayoritaria más cercano, y restringe el radio de generación para no cruzar nunca esa distancia límite. Esto permite crear ejemplos sumamente variados con la garantía matemática de que no invadirán el territorio de los correos legítimos.

**ADASYN + ENN [1].** Este método híbrido combina la creación inteligente de datos de ADASYN con una rigurosa etapa de limpieza algorítmica llamada *Edited Nearest Neighbours* (ENN).

El proceso consta de dos etapas. Primero, ADASYN inyecta nuevos datos sintéticos en las zonas más difíciles de clasificar (aquellas en las que haya ejemplos de las dos clases). Segundo, el algoritmo ENN revisa todo el conjunto de datos ampliado evaluando a los 3 vecinos más cercanos de cada observación. Si detecta que una instancia tiene la mayoría de vecinos pertenecientes a la clase contraria, asume que es un dato que confundirá al clasificador y lo elimina por completo. Se espera que estos dos procesos entreguen un nuevo conjunto de datos sobremuestreado y con nuevos ejemplos que no van a perjudicar al clasificador.

## 2.2 Modelos de Clasificación

Una vez que los datos fueron debidamente preprocesados y balanceados, se procedió a entrenar y evaluar tres modelos clásicos de aprendizaje automático supervisado [3, 2, 5]. Para mantener el enfoque en el análisis del balanceo de datos, se utilizaron las configuraciones pre-determinadas de los algoritmos (provistas por la librería `scikit-learn` [12] y detalladas en las Tablas 2.1 y 2.2), a excepción de un ajuste particular en uno de los modelos que se detalla más adelante.

Tabla 2.1: Parámetros predeterminados utilizados en los clasificadores basados en árboles (scikit-learn).

| Parámetro                                                        | Árbol de Decisión | Bosque Aleatorio                    |
|------------------------------------------------------------------|-------------------|-------------------------------------|
| Criterio de partición ( <code>criterion</code> )                 | <code>gini</code> | <code>gini</code>                   |
| Cantidad de árboles ( <code>n_estimators</code> )                | -                 | 100                                 |
| Profundidad máxima ( <code>max_depth</code> )                    | Ninguna           | Ninguna                             |
| Muestras mínimas para dividir ( <code>min_samples_split</code> ) | 2                 | 2                                   |
| Muestras mínimas por hoja ( <code>min_samples_leaf</code> )      | 1                 | 1                                   |
| Características por división ( <code>max_features</code> )       | Todas             | Raíz cuadrada ( <code>sqrt</code> ) |

Tabla 2.2: Parámetros predeterminados utilizados en la Máquina de Soporte Vectorial (scikit-learn).

| Parámetro                                         | Valor (SVM)                   |
|---------------------------------------------------|-------------------------------|
| Regularización ( <code>C</code> )                 | <code>1.0</code>              |
| Tipo de Kernel ( <code>kernel</code> )            | <code>rbf</code>              |
| Coefficiente Kernel ( <code>gamma</code> )        | <code>scale</code>            |
| Pesos de las clases ( <code>class_weight</code> ) | Ninguno ( <code>None</code> ) |

**Árboles de Decisión [3].** El aprendizaje por Árboles de Decisión es un método no paramétrico que funciona creando divisiones jerárquicas mediante una secuencia de preguntas lógicas del tipo (*if-else*).

El objetivo de este algoritmo es encontrar la partición que separe la clase positiva de la negativa de la forma más limpia posible. Cada vez que el árbol hace una pregunta sobre una característica (por ejemplo, evaluar si un valor supera cierto umbral), el conjunto de datos se divide en sub-nodos. El modelo evaluará estas divisiones buscando alcanzar nodos puros; es decir, hojas finales que contengan exclusivamente instancias de una sola clase.

Para medir la bondad de cada partición y elegir la mejor pregunta, el algoritmo minimiza funciones de impureza, como el índice de Gini [3]. Este índice mide la frecuencia con la que un ejemplo sería clasificado incorrectamente si se lo etiquetara de forma aleatoria. Al tratarse de un problema de clasificación binaria, para un nodo  $t$  se calcula simplemente como:

$$\text{Gini}(t) = 1 - (p(\text{spam}|t)^2 + p(\text{legítimo}|t)^2) \quad (2.2)$$

donde  $p(c|t)$  representa la proporción de ejemplos que pertenecen a una de las dos clases  $c$  dentro de ese nodo. El valor del índice es 0 cuando el nodo es puro (todos los correos son de la misma clase) y aumenta a medida que los correos *spam* y legítimos están más mezclados.

**Bosques Aleatorios [2].** El algoritmo de Random Forest (Bosque Aleatorio) implementa un proceso conocido como *Bagging* (*Bootstrap Aggregating*). En lugar de construir un solo árbol, este método construye múltiples árboles independientes. La clave de su éxito radica en el uso de dos procesos aleatorios durante su creación:

1. **Bootstrap:** Cada árbol se entrena con una muestra aleatoria distinta de los correos originales (con reemplazo).
2. **Selección de atributos:** Al momento de realizar las divisiones lógicas, cada árbol solo puede evaluar un subconjunto aleatorio de las variables. Esto genera muchos árboles con distintas evaluaciones que servirán para evaluar en un futuro las nuevas instancias sin el riesgo de que, durante la fase de entrenamiento, el clasificador se haya sobreentrenado.

Finalmente, cuando llega un correo nuevo (representado matemáticamente como un vector de características  $\mathbf{x}$ ), se realiza el proceso de agregación (*Aggregating*): todos los árboles del bosque lo analizan individualmente y emiten su predicción. Para este caso binario, la clase final del correo ( $\hat{y}$ ) se decide por **voto mayoritario**, eligiendo la clase (ya sea *spam* o legítimo) que reciba la mayor cantidad de votos:

$$\hat{y} = \underset{c \in \{spam, legítimo\}}{\operatorname{argmax}} \sum_{a=1}^A \mathbb{I}(\hat{y}_a(\mathbf{x}) = c) \quad (2.3)$$

donde  $A$  es la cantidad total de árboles en el bosque y  $\mathbb{I}$  es una función indicadora que suma un voto (1) cada vez que un árbol  $a$  vota por una clase en particular  $c$ . Este enfoque combinado permite compensar los errores individuales y logra un modelo predictivo más estable.

**Máquinas de Soporte Vectorial (SVM) [5].** Este algoritmo aborda la clasificación buscando el hiperplano óptimo que produzca el mayor espacio posible entre las clases. A diferencia de otros métodos, la SVM no elige cualquier separación, sino que busca un **Clasificador de Margen Maximal**; es decir, el hiperplano que mejor separe los correos *spam* de los legítimos basándose exclusivamente en las instancias más críticas (los vectores de soporte).

Para lidiar con correos atípicos (*outliers*) y evitar el sobreajuste, la librería **scikit-learn** [12] implementa por defecto un **margen suave**. Siguiendo los principios de este algoritmo, se sacrifica la idea de una separación perfecta (permitiendo ciertas penalidades o errores durante el entrenamiento) para lograr una mayor robustez de clasificación general. Matemáticamente, la decisión de clasificar un nuevo correo  $\mathbf{x}$  se toma evaluando su posición respecto a la frontera mediante la siguiente función:

$$f(\mathbf{x}) = \operatorname{sign} \left( \sum_{i=1}^n \alpha_i y_i \mathcal{K}(\mathbf{x}_i, \mathbf{x}) + b \right) \quad (2.4)$$

donde  $\mathcal{K}(\mathbf{x}_i, \mathbf{x})$  es la función Kernel que calcula la similitud geométrica entre el correo nuevo y los vectores de soporte del entrenamiento,  $\alpha_i$  son unos coeficientes que le otorgan peso exclusivamente a esos vectores críticos,  $y_i$  son las etiquetas conocidas de las clases y  $b$  determina el desplazamiento de la frontera. Si el resultado de la función  $f(\mathbf{x})$  es positivo, el modelo asigna el correo a la clase positiva; si es negativo, a la clase negativa.

### Optimización de Hiperparámetros mediante Grid Search

Para garantizar el máximo rendimiento de la Máquina de Soporte Vectorial, no se utilizaron valores arbitrarios para su configuración interna. Por el contrario, se implementó una búsqueda exhaustiva en cuadrícula (*Grid Search*) combinada con validación cruzada.

Este método automatizado entrena y evalúa el modelo múltiples veces, probando sistemáticamente todas las combinaciones posibles de un conjunto de hiperparámetros predefinidos. El objetivo de esta validación cruzada es encontrar la configuración exacta que minimice el error de clasificación general. En la Tabla 2.3 se detallan los valores probados para cada hiperparámetro durante la fase de optimización.

Tabla 2.3: Grilla de hiperparámetros evaluados y ajustados para el modelo SVM.

| Parámetro                               | Valores Evaluados                   |
|-----------------------------------------|-------------------------------------|
| C (Parámetro de regularización)         | 0,001, 0,01, 0,1, 1, 10, 15, 20, 25 |
| kernel (Tipo de frontera)               | linear, poly, rbf, sigmoid          |
| $\gamma$ (gamma) (Influencia local)     | scale, auto, 0,001, 0,01, 0,1, 1    |
| degree (Grado del polinomio)            | 2, 3, ..., 10                       |
| Umbral de Decisión ( <i>Threshold</i> ) | 0,5 (Por defecto), 0,2 (Optimizado) |

Finalmente, luego de la búsqueda iterativa, se concluyó que los mejores parámetros para este conjunto de datos en particular resultaron ser idénticos a los provistos por la librería por defecto. Tras comprobar esto, se optó por ajustar manualmente el umbral de decisión a 0,2, volviéndolo mucho más riguroso con los correos no deseados. Esta decisión se fundamenta en que es preferible que el usuario tenga que buscar esporádicamente en su carpeta de *spam* un correo de un aviso de pago extraviado, a que, por mantener un filtro permisivo, caiga víctima de un ataque de *phishing*.

## Capítulo 3



## Resultados

En este capítulo se presentan los resultados de los experimentos realizados. Para poner a prueba la efectividad de los generadores de datos sintéticos, se seleccionaron tres algoritmos clásicos de aprendizaje automático: Árbol de Decisión, *Random Forest* y Máquinas de Soporte Vectorial. A lo largo de las pruebas, se analizó el comportamiento de estos modelos al combinarlos con las diferentes técnicas de generación de datos sintéticos en distintas proporciones. El objetivo se basó en encontrar la combinación óptima de modelo de aprendizaje automático y técnica de sobremuestreo capaz de resolver el desafío técnico que enfrentan los filtros en escenarios reales: aprender a detectar amenazas con alta precisión cuando la mayor proporción de los correos recibidos son legítimos.

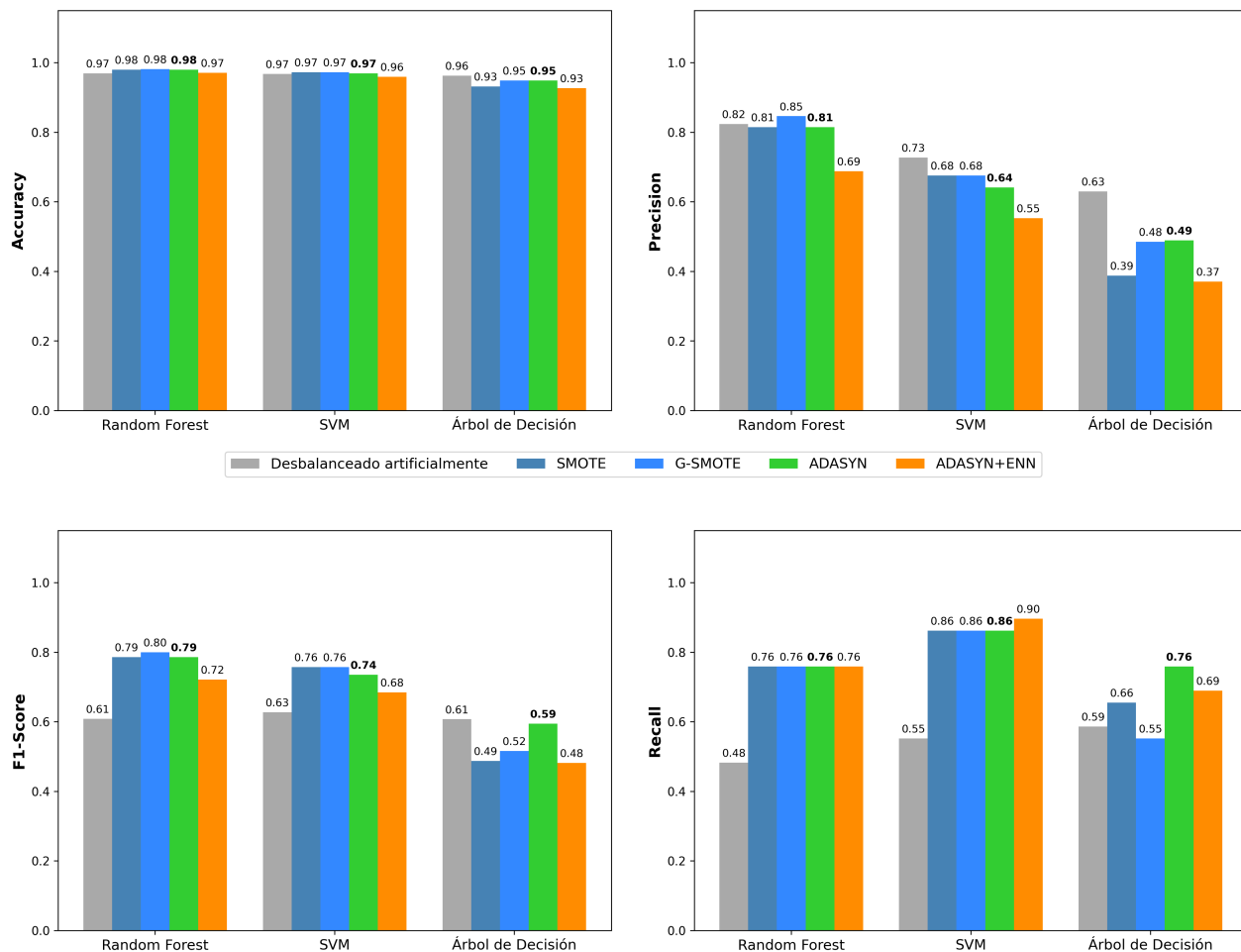


Figura 3.1: Resultados del sobremuestreo al 50/50: *Accuracy*, *Precision*, *F1-Score* y *Recall*.

### 3.1 Resultados de las técnicas de sobremuestreo

Los primeros resultados obtenidos al balancear el conjunto de datos a una proporción 50/50 se pueden observar en la Figura 3.1. Luego de esos resultados se procedió a analizar múltiples niveles de sobremuestreo para encontrar uno que tenga un buen desempeño en las métricas que se usaron para evaluar. Se observó que ajustar la proporción de los datos hasta lograr una relación aproximada del 70 % de correos legítimos y un 30 % de *spam* arrojaba los mejores resultados globales (ver Tabla 3.1). Esta proporción logró compensar las clases adecuadamente: permitió que los algoritmos detecten las amenazas eficientemente, manteniendo al mismo tiempo un control sobre los falsos positivos.

Al evaluar la métrica de *Accuracy*, se evidencia que esta se mantiene elevada en todos los escenarios (variando entre el 95 % y el 98 %). Por este motivo, el *Accuracy* fue descartado como métrica para evaluar la eficiencia del sobremuestreo.

En cuanto a la *Precision*, por un lado el método G-SMOTE, gracias a su restricción geométrica limitando la generación de ejemplos cerca de instancias de la clase mayoritaria al momento del sobremuestreo, resultó ser el más seguro, logrando elevar la *Precision* del Random Forest

hasta un 0,86. En contraparte, la incorporación de la fase de limpieza con ADASYN+ENN demostró una caída en los resultados, eliminando instancias en zonas con las clases mezcladas y provocando una caída generalizada en la *Precision* de todos los modelos.

Tabla 3.1: Métricas obtenidas mediante el clasificador SVM (ADASYN) justificando la proporción 70/30.

| Proporción de Clases | <i>Precision</i> | <i>Recall</i> | <i>F1-Score</i> |
|----------------------|------------------|---------------|-----------------|
| 50/50                | 0,5556           | 0,8621        | 0,6757          |
| 60/40                | 0,6098           | 0,8621        | 0,7143          |
| <b>70/30</b>         | <b>0.6757</b>    | <b>0.8621</b> | <b>0.7576</b>   |
| 80/20                | 0,6471           | 0,7586        | 0,6984          |

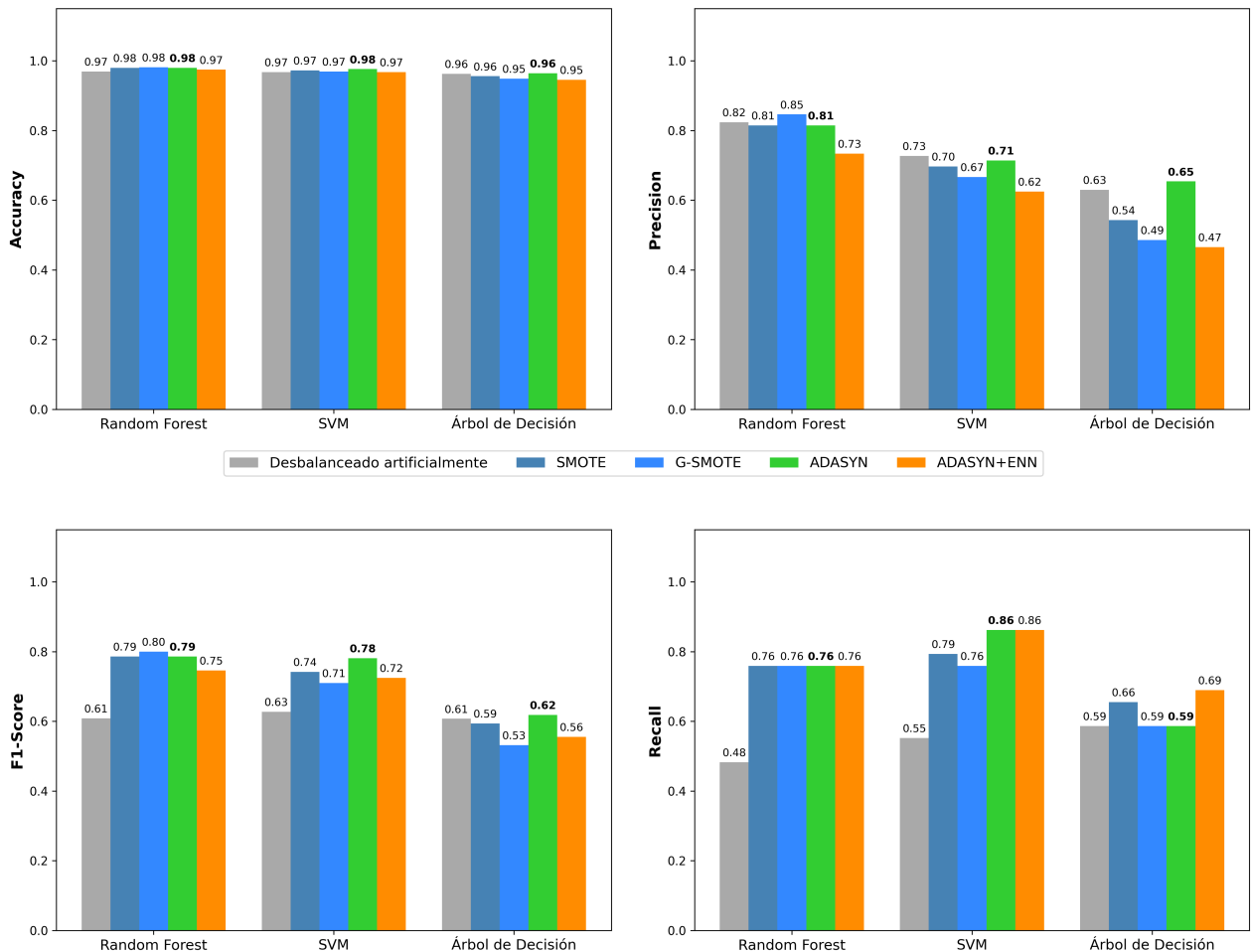


Figura 3.2: Comparación final de métricas utilizando la proporción óptima (70/30).

Por otra parte, el *Recall* es la métrica más importante en este análisis ya que mide cuántos

correos *spam* fueron bloqueados por el filtro y enviados a la bandeja de correo no deseado. El impacto más significativo se observó al aplicar la técnica ADASYN, tanto en su aproximación original como con la limpieza posterior con ENN (ver Figura 3.2).

Luego de las pruebas realizadas en este escenario, se concluye que no existe un clasificador perfecto. La elección del mejor modelo depende de las prioridades y del objetivo principal que se busque. En base a los resultados obtenidos, se destacan dos enfoques principales:

Por un lado, si la prioridad es evitar a toda costa que un correo legítimo termine en la carpeta de *spam*, la mejor combinación demostró ser el Random Forest con clases compensadas mediante la técnica G-SMOTE. G-SMOTE destaca gracias a su restricción geométrica, permitiendo que el Random Forest alcance una *Precision* de 0,84 y el mejor *F1-Score* del estudio (0,80). Si bien este modelo sacrifica levemente su capacidad de identificar todos los correos basura con un *Recall* de 0,75, este clasificador evitará de una mejor manera que se pierda un correo legítimo de importancia para el usuario.

En contraparte, si el objetivo central es la seguridad y se busca maximizar la captura de correos *spam*, la Máquina de Soporte Vectorial (SVM) entrenada con ADASYN (calibrada con un umbral de decisión más riguroso de 0,2) es la ganadora. Esta configuración logra incrementar el Recall hasta un 0,86, detectando la inmensa mayoría de las amenazas. A cambio de esta alta sensibilidad a la hora de detectar *spam*, el modelo presenta una *Precision* más baja (0,71), lo que implica un incremento en los falsos positivos; igualmente, el modelo logra sostener un buen *F1-Score* de 0,78.

## 3.2 Comparación de datos sintéticos frente a originales

Finalmente, se comparó el rendimiento del método de sobremuestreo contra el conjunto de datos *SpamBase* original. Este conjunto contiene aproximadamente un 40 % de correos *spam*. Al entrenar la Máquina de Soporte Vectorial (SVM) con estos datos sin alterar, se logró un *Recall* de 0,89.

Al inicio de las pruebas, se redujo artificialmente la cantidad de *spam* al 5 % para simular un escenario real desfavorable, lo que lógicamente hizo caer la capacidad de detección del modelo. Sin embargo, al aplicar el sobremuestreo con ADASYN para llevar la proporción a un 70/30, el modelo recupera casi por completo su capacidad de detectar *spam*, alcanzando un *Recall* de 0,86 (ver Figura 3.3).

Esta comparación confirma que las técnicas de sobremuestreo tienen un desempeño favorable, permitiendo que un modelo con la mayoría de los ejemplos generados sintéticamente se acerque al rendimiento del conjunto de datos original y mejorando considerablemente sus números respecto al conjunto de datos desbalanceado artificialmente.

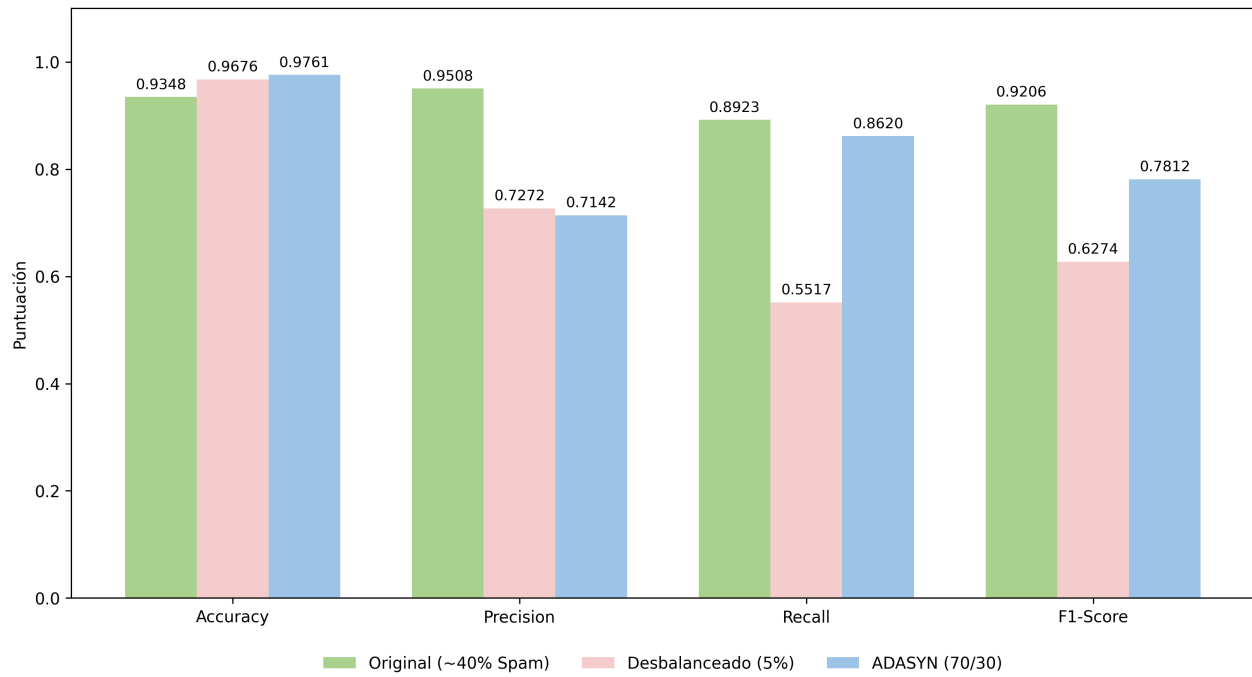


Figura 3.3: Comparación de métricas: Conjunto Original vs. Sobremuestreo ADASYN (70/30).

## Capítulo 4

# Conclusiones

Tras las pruebas realizadas, se concluye que no existe una técnica de sobremuestreo perfecta sino que depende de lo que se priorice:

Si el objetivo principal es garantizar que los correos legítimos no sean bloqueados accidentalmente, la combinación de *Random Forest* con el método G-SMOTE demostró ser la más efectiva. Esta configuración logró el *F1-Score* más alto del estudio (0,80) y una excelente precisión, minimizando la pérdida de correos importantes para el usuario.

Cuando se quiere detectar la mayor cantidad de correos maliciosos posibles a cambio de algún falso positivo, ADASYN combinado con Máquinas de Soporte Vectorial (SVM) es la combinación deseada. Este enfoque alcanzó un *Recall* de 0,86, identificando la inmensa mayoría de las amenazas.

Finalmente, la comparación contra un conjunto de datos compuesto por instancias reales en su totalidad demostró buenos resultados por parte del conjunto sobremuestreado. Las técnicas aplicadas, especialmente ADASYN, permitieron que un modelo que en un principio estaba fuertemente desbalanceado recuperara en gran medida su capacidad de detectar *spam*, acercando su rendimiento al de un modelo entrenado con datos originales abundantes. Esto muestra que las técnicas de sobremuestreo son una herramienta robusta y eficaz a la hora de trabajar con conjuntos de datos con cantidad de ejemplos dispares entre sus clases.

# Bibliografía



- [1] Gustavo EAPA Batista, Ronaldo C Prati, and Maria Carolina Monard. A study of the behavior of several methods for balancing machine learning training data. *ACM SIGKDD explorations newsletter*, 6(1):20–29, 2004.
- [2] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.
- [3] Leo Breiman, Jerome Friedman, Charles J Stone, and Richard A Olshen. *Classification and regression trees*. CRC press, 1984.
- [4] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. Smote: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16:321–357, 2002.
- [5] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine learning*, 20(3):273–297, 1995.
- [6] Emmanuel Gbenga Dada, Joseph Stephen Bassi, Haruna Chiroma, and Shafi'i Muhammad Abdulhamid. Machine learning for email spam filtering: review, approaches and open research problems. *Heliyon*, 5(6):e01802, 2019.
- [7] Georgios Douzas and Fernando Bacao. Geometric smote a geometrically enhanced drop-in replacement for oversampling. *Information sciences*, 501:118–135, 2019.
- [8] ESET Latinoamérica. Estafas más comunes en mercado libre y mercado pago en el 2023. <https://www.eset.com/py/acerca-de-eset/sala-de-prensa/comunicados-de-prensa/articulos-de-prensa/estafas-mas-comunes-en-mercado-libre-y-mercado-pago-en-el-2023/>, 2023. Comunicado de prensa sobre incidentes de ciberseguridad en la región.
- [9] Europol. World's most dangerous malware emotet disrupted through global action. <https://www.europol.europa.eu/media-press/newsroom/news/world's-most-dangerous-malware-emotet-disrupted-through-global-action>, 2021. Comunicado de prensa oficial de la agencia policial europea.
- [10] Alberto Fernández, Salvador Garcia, Francisco Herrera, and Nitesh V Chawla. Smote for learning from imbalanced data: progress and challenges, marking the 15-year anniversary. *Journal of artificial intelligence research*, 61:863–905, 2018.

- [11] Haibo He, Yang Bai, Edwardo A Garcia, and Shutao Li. **Adasyn**: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)*, pages 1322–1328. IEEE, 2008.
- [12] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in **python**. *Journal of machine learning research*, 12(Oct):2825–2830, 2011.
- [13] Brad Templeton. Reaction to the dec spam of 1978. <https://www.templetons.com/brad/spamreact.html>. Registro histórico de las reacciones al primer envío masivo en ARPANET.
- [14] WIRED Staff. The spam that started it all. *WIRED Magazine*, Apr 1999. Artículo sobre el caso Green Card en Usenet.